

Navigation

Đài Loan có mật độ núi cao dày đặc, với hơn 250 đỉnh núi cao hơn 3,000 mét. A-Ming dự định xây dựng mạng lưới robot vận chuyển hàng hóa tự động bằng cáp treo ở Dải núi Trung tâm của Đài Loan để vận chuyển bưu kiện giữa các làng mạc vùng sâu vùng xa.

Trong kế hoạch của A-Ming, mỗi mạng lưới bao gồm N nút, được đánh số từ 0 đến $N - 1$. Trong số các nút này, **chính xác** $K = 6$ nút là các trạm điện và $N - K$ nút còn lại là các làng. Các nút được kết nối bằng $N - 1$ dây cáp **hai chiều**, được đánh số từ 0 đến $N - 2$. Với mỗi i ($0 \leq i \leq N - 2$), dây cáp i kết nối các nút $U[i]$ và $V[i]$. Mỗi dây cáp kết nối hai nút khác nhau, và mỗi cặp nút được kết nối bởi tối đa một dây cáp. Đảm bảo rằng giữa hai nút bất kỳ đều có thể di chuyển thông qua các dây cáp trong mạng.

Khoảng cách giữa hai nút a và b được ký hiệu là $d(a, b)$ và được định nghĩa như sau:

- Nếu $a = b$ thì $d(a, b) = 0$.
- Trái lại, $d(a, b)$ là số lượng dây cáp tối thiểu cần thiết để di chuyển từ a đến b .

Ta nói rằng **hệ số phân nhánh** của mạng ít nhất là δ nếu mỗi nút trong mạng được kết nối với chính xác 1 hoặc ít nhất δ dây cáp. A-Ming biết một số nguyên dương B mà hệ số phân nhánh của mạng ít nhất là B .

A-Ming đã chuẩn bị 100 loại dây cáp khác nhau, được đánh số từ $0, 1, \dots, 99$. Mỗi dây cáp trong mạng được thiết lập từ một trong những loại dây cáp đó.

Các robot sẽ di chuyển dọc theo dây cáp để thực hiện nhiệm vụ giao hàng. A-Ming muốn cài đặt một chương trình điều hướng giúp robot quay trở lại các trạm điện và tự sạc pin. Tuy nhiên, các robot có bộ nhớ cực kỳ hạn chế. Do đó, khi thiết kế chương trình điều hướng, robot chỉ điều hướng dựa trên số lượng trạm điện $K = 6$, hệ số phân nhánh đảm bảo B và các loại dây cáp được gắn vào vị trí hiện tại của robot.

Khi một robot muốn di chuyển tại một nút u là làng được kết nối với hai hoặc nhiều dây cáp, robot trước tiên sẽ quét các dây cáp được kết nối với u theo một thứ tự bất kỳ. Sau đó, chương trình được cung cấp một danh sách chứa các loại dây cáp theo thứ tự đó. Đối với mỗi dây cáp, chương trình cần đếm số lượng trạm điện sao cho việc đi theo dây cáp đó làm giảm khoảng cách giữa robot và trạm điện.

Cụ thể, gọi $v_1, \dots, v_k$ ($k \geq 2$) là các nút được kết nối trực tiếp với u bằng một dây cáp và c_i ($1 \leq i \leq k$) là loại dây cáp nối nút u và v_i . Chương trình được cung cấp K , B , và danh sách $c_1, c_2, \dots, c_k$. Với mỗi i ($1 \leq i \leq k$), chương trình phải xác định số lượng trạm điện p sao cho $d(v_i, p) < d(u, p)$.

Nhiệm vụ của bạn là xây dựng chiến lược để phân bổ các loại cáp và thiết kế chương trình điều hướng. Điểm số của bài làm của bạn phụ thuộc vào số lượng các loại cáp khác nhau được sử dụng (xem phần Subtask và Chấm điểm để biết thêm chi tiết).

Chi tiết cài đặt

Bạn cần thực hiện hai hàm, một để phân bổ loại cáp và một để cho chương trình của robot.

Hàm bạn cần cài đặt để **phân bổ các loại cáp** là:

```
std::vector<int> construct_network(int N, int K, int B,
                                   std::vector<int> U, std::vector<int> V,
                                   std::vector<int> P)
```

- N : số lượng nút.
- K : số lượng các trạm điện.
- B : hệ số phân nhánh đảm bảo tối thiểu của mạng.
- U, V : các mảng độ dài $N - 1$ mô tả các sự kết nối của dây cáp.
- P : một mảng độ dài $K = 6$ mô tả các nút là trạm điện.
- Hàm này được gọi **tối đa 10 000 lần** cho mỗi trường hợp thử nghiệm.

Hàm này cần trả về một mảng T độ dài $N - 1$:

- Một dây cáp thuộc loại $T[i]$ được sử dụng để kết nối nút $U[i]$ và $V[i]$.
- Mỗi phần tử $T[i]$ phải thoả mãn $0 \leq T[i] < 100$.

Hàm bạn cần cài đặt cho **chương trình của robot** là:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : số lượng các trạm điện.
- B : hệ số phân nhánh đảm bảo tối thiểu của mạng.
- C : danh sách các loại cáp của tất cả dây cáp có kết nối đến **nút làng mà kết nối với ít nhất 2 dây cáp**, theo thứ tự bất kỳ.
- Đảm bảo rằng độ dài của C tối thiểu là B .
- Hàm này được gọi tối đa 100 000 lần với mỗi trường hợp thử nghiệm.

- Hàm điều hướng này có thể không phụ thuộc vào cấu trúc mạng ban đầu; đặc biệt, hệ thống chấm điểm có thể sắp xếp lại tất cả các lệnh điều hướng trên các mạng được xây dựng khác nhau trong cùng một trường hợp thử nghiệm.

Hàm này cần trả về một mảng D :

- Độ dài mảng D phải bằng độ dài mảng C .
- $D[i]$ phải là số lượng các trạm điện mà việc đi theo dây cáp tương ứng với $C[i]$ làm giảm khoảng cách giữa robot và trạm điện.

Lưu ý là trong hệ thống chấm điểm, các hàm `construct_network` và `navigate` được gọi **trong hai tiến trình riêng biệt**.

Ràng buộc

- $K = 6$.
- $K < N \leq 100\,000$.
- Tổng các N qua tất cả lời gọi đến hàm `construct_network` không vượt quá 100 000 với mỗi trường hợp thử nghiệm.
- $0 \leq U[i] < V[i] < N$ với mỗi $0 \leq i \leq N - 1$.
- Luôn có thể di chuyển từ một nút bất kỳ sang một nút bất kỳ khác thông qua các dây cáp.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- Mỗi mảng C là một hoán vị của danh sách các loại cáp được kết nối với một nút làng nào đó, nút này được kết nối với ít nhất 2 cáp.
- Tổng độ dài của mảng C qua tất cả các lời gọi đến hàm `navigate` không vượt quá 200 000 đối với mỗi trường hợp thử nghiệm.

Subtask và Chấm điểm

Subtask	Điểm	Ràng buộc bổ sung
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ với mỗi i sao cho $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

Trong bất kỳ trường hợp thử nghiệm nào, nếu ít nhất một trong các điều kiện sau đây xảy ra, thì điểm số của bài làm của bạn cho trường hợp thử nghiệm đó sẽ là 0 (được thông báo là `Output isn't correct` trong CMS):

- Giá trị trả về của bất kỳ lệnh gọi nào đến `construct_network` đều không hợp lệ.
- Giá trị trả về của bất kỳ lệnh gọi nào đến `navigate` đều không chính xác.

Trái lại, gọi S là số nguyên nhỏ nhất lớn hơn tất cả các số trong mọi mảng T được trả về từ mỗi lệnh gọi `construct_network`. Nói cách khác, S là số nguyên nhỏ nhất sao cho tất cả các mạng được xây dựng chỉ sử dụng các dây cáp loại $0, 1, \dots, S - 1$.

Điểm của bạn cho mỗi subtask phụ thuộc vào S như sau:

Điều kiện	Subtask 1	Subtask 2	Subtask 3 và 4	Subtask 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

Đặc biệt, trong Subtask 1, 2 và 5, bạn sẽ nhận được toàn bộ số điểm nếu $S \leq 5$, và trong Subtask 3 và 4, bạn sẽ nhận được toàn bộ số điểm nếu $S \leq 4$.

Ví dụ

Xét kịch bản với $N = 9$, $K = 6$, và $B = 2$.

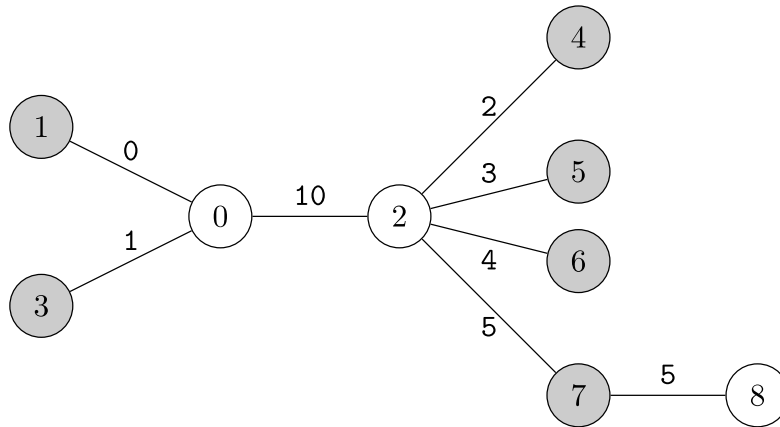
Cấu trúc của mạng được cho bởi $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$. Các trạm điện là $P = [1, 3, 4, 5, 6, 7]$. Xét lời gọi sau đến tiến trình đầu tiên:

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

Giả sử A-Ming xây dựng mạng bằng cách trả về:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

Mạng vừa xây dựng được minh hoạ như sau.



Tiếp theo, trong tiến trình thứ hai, giả sử robot cần điều hướng bắt đầu tại nút 0. Nút 0 được kết nối với các nút 1, 2 và 3. Nếu robot đọc loại dây cáp theo thứ tự này, lệnh gọi sau sẽ được thực hiện:

```
navigate(6, 2, [0, 10, 1])
```

Dựa trên cấu trúc mạng:

- Trạm điện đầu tiên, nút 1, có khoảng cách ngắn nhất đến chính nó. Cụ thể, $0 = d(1,1) < 1 = d(0,1) < 2 = d(2,1) = d(3,1)$.
- Trạm điện thứ hai, nút 3, có khoảng cách ngắn nhất đến chính nó. Cụ thể, $0 = d(3,3) < 1 = d(0,3) < 2 = d(1,3) = d(2,3)$.
- Các trạm điện khác, nút 4, 5, 6, và 7, có các khoảng cách ngắn hơn đến nút 2. Cụ thể, $1 = d(2,u) < 2 = d(0,u) < 3 = d(1,u) = d(3,u)$ với $u = 4, 5, 6$, or 7.

Số lượng các trạm điện tương ứng với khoảng cách giảm khi đi theo các dây cáp (0, 1), (0, 2), và (0, 3) là 1, 4 và 1. Do đó, hàm cần trả về:

```
[1, 4, 1]
```

Lưu ý là hàm `navigate` cũng có thể được gọi cho các hoán vị khác nhau của C . Ví dụ, `navigate(6, 2, [1, 0, 10])` cũng có thể là một lời gọi cho nút 0. Do đó, hàm cần trả về `[1, 1, 4]`.

Giả sử robot cần điều hướng bắt đầu tại nút 2. Xét lời gọi hàm sau:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

Hàm cần trả về:

```
[2, 1, 1, 1, 1]
```

Trong mạng này, hàm `navigate` sẽ **không bao giờ** được gọi cho các nút khác vì:

- nút 1, 3, 4, 5, 6 và 7 và tất cả các trạm điện, và
- nút 8 được nối với duy nhất một dây cáp.

Trong trường hợp thử nghiệm này, các loại dây cáp được sử dụng là 0, 1, 2, 3, 4, 5, và 10. Do đó, $S = 11$ được sử dụng để tính điểm cho trường hợp thử nghiệm này.

Gói đính kèm cho bài toán này chứa hai ví dụ đầu vào và đầu ra khác cho trình chấm mẫu.

Trình chấm mẫu

Trình chấm mẫu gọi cả hai hàm `construct_network` và `navigate` trong cùng một tiến trình. Ngoài ra, khi trình chấm mẫu gọi hàm `navigate`, các loại dây cáp được liệt kê theo thứ tự xuất hiện của chúng trong dữ liệu đầu vào. Cả hai hành vi này đều khác với chương trình chấm điểm thực tế.

Định dạng dữ liệu vào:

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

Mỗi kịch bản theo định dạng sau:

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

Trong mỗi kịch bản, trình chấm mẫu sẽ gọi hàm `navigate` Q lần, lần thứ i gọi với thông tin của nút $X[i - 1]$. Lưu ý là trình chấm mẫu hỗ trợ thiết lập thành các số nguyên khác 6.

Định dạng dữ liệu ra:

Nếu bất kỳ mảng nào được trả về từ `construct_network` hoặc `navigate` không hợp lệ, trình chấm mẫu sẽ dừng và thông báo lý do. Ngược lại, trình chấm mẫu sẽ xuất ra từng mảng D được trả về từ lệnh gọi `navigate` trên một dòng.

Sau khi tất cả các kịch bản đã được xử lý, trình chấm mẫu sẽ in một dòng về sai số chuẩn:

$$S = S'$$

với S' là số nguyên nhỏ nhất lớn hơn tất cả các số trong mọi mảng T được trả về từ mỗi lệnh gọi `construct_network` trong trường hợp thử nghiệm đó.